

ECOM EXPERTS

WithdrawalKit EU — EuroDesist

Manual de usuario e instalación

desistimiento-v2 · app de Shopify · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

4.1 General

4.2 Estados y plazo

4.3 Productos excluidos

4.4 B2B / Profesionales

4.5 Emails

5. Cómo se usa, día a día

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que ya estaba bien

Lo que se ha arreglado en esta revisión

Lo que queda pendiente

9. Ficha técnica

Identidad de la aplicación

Tablas (Prisma / SQLite)

Rutas

Webhooks

Extensiones

Variables de entorno

Planes de facturación

Registros

Copia de trabajo `desistimiento-v2`. Es una **aplicación de Shopify** escrita en Node.js con Remix, no un módulo de PrestaShop: no se instala en ninguna tienda local, se despliega en un servidor propio y se conecta a Shopify.

1. Qué hace y para quién

Desde el 19 de junio de 2026, la Directiva (UE) 2023/2673 obliga a toda tienda en línea europea a poner a disposición del comprador un medio visible y sencillo para ejercer el derecho de desistimiento. No basta con tenerlo escrito en las condiciones de venta: tiene que haber un enlace o un botón que lleve a un formulario que el cliente pueda rellenar y enviar. Esta aplicación es exactamente eso para una tienda de Shopify.

Al cliente le ofrece una página pública dentro de la propia tienda —en <https://la-tienda.com/apps/desistimiento>— donde localiza su pedido con el número y el correo con el que compró, elige qué productos quiere devolver y con qué cantidad, indica un motivo si quiere y firma la declaración del Anexo B del texto refundido de la Ley General para la Defensa de los Consumidores y Usuarios. Al enviarla, la solicitud queda registrada con fecha y hora, que es lo que legalmente cuenta para el plazo de catorce días naturales.

Al comerciante le da un panel dentro del administrador de Shopify con todas las solicitudes recibidas, su estado, el cliente, el pedido y las líneas concretas que se quieren devolver. Desde ahí las va moviendo por los seis estados previstos —recibida, en revisión, aceptada, producto recibido, reembolsada, rechazada— y anota los importes de producto, envío y depreciación para dejar constancia de qué se ha devuelto y por qué. El reembolso real se ejecuta en Shopify, en la página del pedido; la aplicación enlaza directamente a ella pero no mueve dinero.

Está pensada para el comercio pequeño y mediano que vende a consumidores en España y en el resto de la Unión Europea, y que necesita cumplir sin montarse un proceso propio. Los textos legales vienen escritos por defecto con las referencias correctas —artículos 102 a 108 y 103 del TRLGDCU, Directiva 2011/83/UE— y son editables, porque cada comercio tiene sus particularidades y sus exclusiones.

2. Compatibilidad

Requisito	Valor
Plataforma	Shopify (aplicación embebida, distribución App Store)
Versión de la API de Shopify	2026-01 (<code>ApiVersion.January25</code>)
Entorno de ejecución	Node.js <code>>=20.19 <22</code> o <code>>=22.12</code>
Marco de trabajo	Remix 2.16 + Vite 6 + React 18 + Polaris 12 + App Bridge 4
Base de datos	Prisma 6 sobre SQLite (<code>prisma/dev.sqlite</code>)
Permisos que pide	<code>read_customers</code> , <code>read_locales</code> , <code>read_orders</code> , <code>read_products</code> , <code>read_themes</code>
Idiomas	Uno: castellano. Ver la advertencia de abajo
Multi-tienda	Sí: cada tienda tiene su fila en <code>Shop</code> y su propia configuración

Sobre los idiomas, con cuidado. La estructura de datos está preparada para varios idiomas: los textos legales y los datos del empresario se guardan como un diccionario donde la clave es el idioma, y hay valores por defecto escritos en castellano y en inglés. Pero **las pantallas de configuración solo permiten editar la versión castellana y el formulario público solo lee la castellana**. Los planes de pago anuncian siete idiomas; en el código no hay ni selector de idioma ni traducción de la página pública. Es la diferencia más grande entre lo que se promete y lo que hay.

Sobre la base de datos. El fichero `prisma/schema.prisma` tiene la ruta escrita directamente en el código (`url = "file:dev.sqlite"`), no leída de `DATABASE_URL`. En un servidor con contenedores eso significa que la base de datos vive dentro del contenedor y se pierde en cada redespigue, y que no se puede levantar más de una instancia. Antes de poner esto en producción hay que decidir dónde vive el fichero y, si va a haber más de un proceso, cambiar a un motor de servidor.

3. Instalación

La aplicación no se instala como un módulo: se despliega en un servidor y se conecta con Shopify desde el panel de Partners.

Lo que hay que tener en el servidor. Node en una de las versiones admitidas, un dominio con certificado válido —el configurado es `https://desistimiento.gmartos.es`— y un proceso que mantenga la aplicación viva. La secuencia es la habitual de un proyecto Remix: instalar dependencias, `npm run setup` (que ejecuta `prisma generate` y `prisma migrate deploy`), `npm run build` y `npm start`. Hay también un `Dockerfile` que hace lo mismo dentro de un contenedor con `node:18-alpine` — conviene saber que esa imagen es **anterior** a la versión de Node que declara el propio `package.json`, así que o se sube la imagen o se usa el despliegue directo.

Variables de entorno necesarias, todas fuera del código: `SHOPIFY_API_KEY`, `SHOPIFY_API_SECRET`, `SHOPIFY_APP_URL`, `SCOPES` y, opcionalmente, `SHOP_CUSTOM_DOMAIN`. A partir de esta revisión hay dos más,

ambas opcionales: `WK_DEBUG=1` para activar el registro de depuración del formulario público, y `SHOPIFY_BILLING_TEST=false` para que los planes se cobren de verdad (ver el apartado 7).

Lo que crea. Al arrancar, Prisma crea siete tablas: `Session`, `Shop`, `ShopConfig`, `DesistRequest`, `DesistRequestDetail`, `DesistLog` y `WebhookSeen`. La fila de `Shop` y la de `ShopConfig` de cada comercio no se crean al instalar, sino la primera vez que se abre la configuración o el formulario público: la función `ensureShop()` las da de alta sobre la marcha con catorce días de prueba.

Lo que hay que activar a mano en la tienda, y es obligatorio. La aplicación instala tres extensiones, pero ninguna se enciende sola:

1. **Icono flotante** (bloque incrustado de tema). Se activa desde el editor de temas. Pinta un botón redondo fijo en una esquina que lleva al formulario.
2. **Enlace para el pie de página** (bloque de tema). Se añade a la sección donde se quiera, normalmente el pie.
3. **Botón en la cuenta del cliente** (extensión de interfaz). Aparece en el detalle de un pedido y en el listado de pedidos de la cuenta del cliente.

El panel principal de la aplicación tiene un aviso en rojo con dos botones que abren directamente el editor de temas en el sitio correcto. Si no se activa ninguna de las tres, la página del formulario existe pero **no hay ningún enlace hacia ella**, que es justo lo que la Directiva exige.

Qué pasa al desinstalar. Shopify envía el webhook `app/uninstalled`. La aplicación marca la tienda con la fecha de desinstalación y borra sus sesiones, pero **conserva las solicitudes y la configuración**. Es deliberado: si el comercio reinstala, se encuentra su histórico. El borrado real solo ocurre con el webhook `shop/redact`, que Shopify manda cuarenta y ocho horas después de la desinstalación definitiva, y ese sí elimina solicitudes, registros, configuración y la propia tienda.

4. Configuración

No hay capturas de pantalla en este manual. Esta revisión se ha hecho leyendo el código, sin instalar la aplicación en una tienda y sin levantar el servidor, que era la condición del encargo. Las pantallas se describen a partir de los componentes de Polaris que las construyen.

La configuración está en `Configuración` del menú lateral y son cinco pestañas dentro de una misma página. Cada pestaña tiene su propio botón de guardar y guarda solo sus campos.

4.1 General

Activación. Cuatro casillas. «Activar botón de desistimiento en la tienda» y «Mostrar icono flotante» se guardan correctamente, pero **hoy no cambian nada**: el formulario público no consulta ninguna de las dos y el icono flotante se enciende y se apaga desde el editor de temas, no desde aquí. «Mostrar solo en países UE/EEA» sí funciona, pero tampoco desde aquí: la comprobación está escrita en el bloque de tema del icono flotante, que compara el país de la tienda contra la lista de los veintisiete. «Activar registro de depuración» se guarda y no se usa; el registro de depuración se activa con la variable de entorno `WK_DEBUG`.

Coste de devolución. Elige quién paga el porte de vuelta, el cliente o el comerciante. Es un dato informativo que se guarda; conviene que coincida con lo que digan las condiciones de venta, porque el artículo 108 del

TRLGDCU obliga a haber informado al consumidor de que corre a su cargo, y si no se le informó, lo paga el comerciante.

Datos del empresario (Anexo B). Nombre o razón social, dirección, teléfono y correo. Estos cuatro campos **sí salen en la página pública**, en el recuadro «Datos del comerciante (destinatario del desistimiento)» y en la pantalla de confirmación. El Anexo B del TRLGDCU exige que el consumidor sepa a quién dirige su declaración, así que hay que rellenarlos con los datos reales del titular del comercio, los mismos del aviso legal.

Texto legal del desistimiento. Un área de texto grande con el texto por defecto ya escrito, que reproduce el modelo del Anexo A: los catorce días, cuándo empiezan a contar, cómo se comunica y cómo se reembolsa. Se puede sustituir entero. Sale en la página pública, en el recuadro «Información legal del desistimiento». Si se deja vacío, sale vacío: el valor por defecto solo se aplica la primera vez que se crea la configuración.

Apariencia del botón. Forma —redondeada, píldora o cuadrada— y dos colores en hexadecimal, el de marca y el de acento. Afectan a la página pública del formulario: el color de marca es el de los botones y el borde de los campos al enfocarlos; el de acento, el de los enlaces. Desde esta revisión, cualquier valor que no sea un hexadecimal válido se rechaza al guardar y se cae al color por defecto.

4.2 Estados y plazo

Días exactos. El plazo en días naturales, entre 7 y 365. El valor legal en la Unión Europea es 14 y no se debe bajar de ahí; subirlo es legítimo y es una decisión comercial —hay comercios que dan 30 como argumento de venta—. El campo se guarda bien.

Inicio del cómputo del plazo. Cuatro opciones: fecha de creación del pedido, cuando se cumple el pedido (sale del almacén), cuando se marca como entregado, o cuando el pedido recibe una etiqueta personalizada que se escribe en el campo de abajo. La opción correcta en derecho es **la entrega**: el artículo 104 del TRLGDCU cuenta desde que el consumidor adquiere la posesión material del bien.

Advertencia importante sobre esta pestaña. Los dos valores se guardan y el listado de solicitudes enseña «Plazo configurado: N días» en su subtítulo. Pero **el plazo no se aplica en ninguna parte**. El formulario público no comprueba si el pedido está dentro de plazo antes de aceptar la solicitud: la función que calcula la fecha límite (`deadlineFor`) está escrita y es correcta, pero nadie la llama. Un cliente puede presentar un desistimiento de un pedido de hace un año y le entrará al comerciante como cualquier otro. Hay que revisarlo a mano.

4.3 Productos excluidos

Cuatro listas separadas por comas: identificadores de producto, identificadores de colección, nombres de vendedor o marca, y etiquetas de producto. Cualquiera de las cuatro sirve para dejar fuera del desistimiento los supuestos del artículo 103 del TRLGDCU: productos personalizados, precintados que no se pueden devolver por higiene, perecederos, contenidos digitales ya descargados.

Debajo, el texto legal de exclusiones, con el texto del artículo 103 escrito por defecto. Ese texto **sí sale** en la página pública, en el recuadro «Productos excluidos del desistimiento».

Misma advertencia que la anterior. Las cuatro listas se guardan y el texto se publica, pero **las exclusiones no se aplican**. La función que decide si una línea de pedido está excluida (`isLineExcluded`) está escrita, está

bien y no la llama nadie. El formulario público ofrece al cliente todas las líneas del pedido, incluidas las excluidas. La lista sirve hoy como documentación de cara al cliente y como recordatorio para el comerciante al revisar la solicitud.

4.4 B2B / Profesionales

El derecho de desistimiento protege al consumidor final, no al profesional. Esta pestaña define cómo se detecta que un pedido es de empresa: por el campo «empresa» relleno, por llevar NIF/CIF/VAT, o por una lista de etiquetas de cliente. Y qué hacer entonces: bloquear la solicitud o permitir la igualmente como cortesía comercial. Debajo, el texto legal que explica al cliente por qué su pedido no tiene derecho al desistimiento.

Y la misma advertencia, por tercera vez. La detección (`isB2B`) está escrita y probada, y nadie la llama. La columna `isB2B` de la base de datos **siempre vale falso**, así que la etiqueta «B2B» que el listado y el detalle saben pintar no aparece nunca. Ni se bloquea ni se avisa.

4.5 Emails

Cinco casillas —confirmación de pedido, factura, confirmación de envío, actualización de envío, entrega confirmada—, la posición del aviso (antes o después del contenido) y el texto del aviso legal a inyectar.

Esta pestaña no envía ni modifica ningún correo. Lo que hace de verdad es guardar el texto y enseñar debajo un fragmento de código Liquid **para que el comerciante lo copie y lo pegue a mano** en las plantillas de notificación de Shopify (Configuración → Notificaciones). Las casillas son una lista de tareas para acordarse de en cuáles se ha pegado. Está bien que sea así —Shopify no deja a una aplicación reescribir las notificaciones sin más—, pero el nombre de la pestaña engaña.

5. Cómo se usa, día a día

El panel principal. Cuatro contadores en cabecera: total de solicitudes, pendientes —las que están en recibida, en revisión o aceptada—, reembolsadas y rechazadas. Encima, el aviso rojo con los botones que abren el editor de temas para activar el enlace. Ese aviso no se puede quitar y es deliberado: sin enlace no hay cumplimiento.

El listado de solicitudes. Una tabla con el número de pedido, el cliente y su correo, la fecha, el estado con su color, si es B2B, el número de líneas y el importe. Arriba, un buscador y un filtro por estado. Conviene saber dos cosas: el filtrado y la búsqueda se hacen **en el navegador**, sobre lo ya cargado, y la consulta al servidor trae como mucho doscientas solicitudes. Un comercio con más de doscientas verá solo las doscientas más recientes, y el buscador no encontrará las anteriores.

La ficha de una solicitud. Se abre pulsando el número de pedido. Arriba, el estado y los datos del cliente, la fecha, el motivo y las observaciones que escribió. Debajo, los productos concretos que pide devolver con sus cantidades. Luego dos formularios y un historial:

Cambiar estado. Un desplegable con los seis estados y una nota opcional. Cada cambio se escribe en el historial con quién lo hizo (el correo del empleado de Shopify), de qué estado a cuál y la nota. Ese historial es la prueba de que se atendió la solicitud y cuándo, que es lo que hay que poder enseñar si llega una reclamación. Al pasar a «Reembolsada» se guarda además la fecha del reembolso.

Importes a reembolsar. Tres campos —producto, envío estándar y depreciación— y una nota para justificar la depreciación. El total se calcula a la vista: producto más envío menos depreciación. **Aquí no se mueve dinero.** El botón «Abrir reembolso en Shopify» lleva a la página de reembolso del pedido real, y ahí es donde se ejecuta. Estos campos son el registro de auditoría de lo acordado.

Sobre el envío: el artículo 107 del TRLGDCU obliga a devolver también los gastos de envío **estándar** de ida, no los de una modalidad más cara que el cliente eligiera. Y sobre la depreciación, el artículo 108 permite descontar la disminución de valor por un uso distinto al necesario para comprobar el producto —probárselo, sí; usarlo un mes, no—, pero solo si se informó al consumidor del derecho de desistimiento. Si no se le informó, no se puede descontar nada.

El flujo normal. Llega una solicitud en estado «Recibida». Se comprueba contra el pedido real que está en plazo y que el producto no está excluido —a mano, porque la aplicación no lo hace—. Se pasa a «En revisión» mientras se decide, a «Aceptada» cuando se le manda al cliente cómo devolver, a «Producto recibido» cuando llega, y a «Reembolsada» cuando se ha ejecutado el reembolso en Shopify. Si no procede, «Rechazada» con la nota explicando por qué, que es lo que sirve de defensa.

El plazo del comerciante. Catorce días naturales desde que recibe la comunicación para devolver el dinero, aunque el artículo 107.3 permite retenerlo hasta haber recibido el producto de vuelta o la prueba de su envío.

6. Qué ve el cliente en la tienda

El acceso. Según lo que se haya activado: un botón redondo fijo en una esquina, un enlace en el pie de página, o un aviso azul con un botón en la cuenta del cliente, tanto en el listado de pedidos como en el detalle de uno. Los tres llevan al mismo sitio.

Paso 1, localizar el pedido. La página de entrada explica el derecho en dos líneas y pide dos datos: el número o referencia del pedido y el correo con el que compró. Debajo, el «cómo funciona» en cuatro pasos, el texto legal del desistimiento, el de las exclusiones y los datos del comerciante.

Paso 2, el formulario del Anexo B. Si el pedido existe **y el correo coincide con el del pedido**, aparece el formulario: nombre completo, teléfono opcional, correo de contacto, motivo del desistimiento y observaciones. Debajo, las líneas del pedido real traídas de Shopify, cada una con su casilla y su cantidad ajustable, para poder desistir solo de parte. Y la casilla de la declaración, con el texto formal de la comunicación de desistimiento.

El motivo se pide con una advertencia honesta escrita en la propia pantalla: legalmente el consumidor **no tiene que justificar nada** (artículo 104), y el dato se pide solo con fines estadísticos. Está bien resuelto.

Paso 3, la confirmación. Una referencia interna de doce caracteres, el resumen de lo enviado y los tres próximos pasos. Si el cliente vuelve a enviar el mismo formulario, o vuelve a buscar el mismo pedido, se le enseña la solicitud que ya tiene con su estado y su fecha en vez de crear una segunda.

Lo que el cliente NO recibe, y conviene saberlo. No hay correo de confirmación y no hay PDF del Anexo B. En esta revisión se han corregido las dos frases de la página que se lo prometían, porque prometer al consumidor un justificante que no llega es peor que no prometerlo. El justificante hoy es la pantalla de

confirmación con su referencia. Si el comercio necesita mandar un acuse por correo —y es lo razonable—, hoy hay que hacerlo a mano desde el panel.

7. Preguntas frecuentes y problemas

He instalado la aplicación y en mi tienda no aparece nada. Es lo normal: ninguna de las tres extensiones se enciende sola. Hay que ir al editor de temas y activar el icono flotante, el enlace del pie, o los dos. Los botones del aviso rojo del panel principal abren el editor en el sitio exacto.

Un cliente dice que no encuentra su pedido. Tienen que coincidir las dos cosas: el número de pedido **y** el correo con el que se hizo la compra. El correo se compara con el del pedido en Shopify, no con el que el cliente tenga en su cuenta ahora. Si compró como invitado con otro correo, o si el pedido se creó a mano en el administrador sin correo, no lo encontrará. El número se puede escribir con o sin almohadilla.

¿Puede alguien meter un desistimiento de un pedido que no es suyo? Desde esta revisión, no: la aplicación relee el pedido en Shopify y exige que el correo coincida con el del pedido. Antes de este arreglo sí se podía; está explicado en el apartado 8.

El plazo que he configurado no se respeta. Correcto, y es la limitación más importante de esta versión. Ni el plazo, ni las exclusiones, ni las reglas B2B se aplican en el formulario público. Hay que comprobarlo a mano en cada solicitud. Está detallado en los apartados 4.2, 4.3, 4.4 y 8.

El globo con el número del icono flotante nunca aparece. También correcto. El JavaScript del tema pide un número a `/apps/desistimiento/count` y esa dirección devuelve la página HTML del formulario, no un número. El error se traga en silencio.

Me he suscrito a un plan y no me han cobrado nada. Es el comportamiento actual: todas las suscripciones se crean en modo de prueba. Para cobrar de verdad hay que definir `SHOPIFY_BILLING_TEST=false` en el servidor. Ver el apartado 8.

No veo las solicitudes antiguas. El listado trae como mucho doscientas, las más recientes, y el buscador solo mira entre esas. No hay paginación.

¿La aplicación reembolsa el dinero? No. Registra los importes acordados y enlaza a la página de reembolso del pedido en Shopify, donde el comerciante lo ejecuta.

¿Y si el cliente quiere devolver solo un artículo de tres? Se puede: en el formulario desmarca los que no devuelve y ajusta la cantidad de los que sí.

He desinstalado la aplicación, ¿se han borrado los datos? No de inmediato. Se borran las sesiones y se marca la tienda como desinstalada. Las solicitudes se conservan por si se reinstala, y se eliminan de verdad cuando Shopify envía `shop/redact`, unas cuarenta y ocho horas después de la desinstalación definitiva.

8. Seguridad

Se ha revisado el código completo de esta copia: el arranque del SDK, las veintiuna rutas de Remix, los dos servicios, el manejador del formulario público, las tres extensiones, el esquema de Prisma y los ficheros de configuración del proyecto. Se ha buscado en concreto lo propio de una aplicación de Shopify sobre Node y

Remix: la verificación HMAC de los webhooks, la validación de la sesión embebida, la firma del proxy, los secretos dentro del paquete, la inyección en consultas y en el HTML, las cabeceras de CORS y de caché, y los datos personales escritos en los ficheros de registro. Se ha comprobado además, uno por uno, si se repetían aquí los once fallos que salieron en la revisión de su proyecto hermano `desistimiento-shopify`.

La revisión se ha hecho **leyendo el código**, sin instalar dependencias, sin levantar el servidor y sin abrir la aplicación en una tienda, que era la condición del encargo. Lo que se afirma aquí está verificado en el código; lo que no se ha podido verificar se dice.

Lo que ya estaba bien

- **Los webhooks se autentican de verdad.** `authenticate.webhook()` comprueba la firma HMAC antes de dar por buena la carga útil, y la tabla `WebhookSeen` descarta los reenvíos duplicados que Shopify hace por diseño.
- **Las cuatro tareas de cumplimiento del RGPD están implementadas y hacen lo que dicen.** `customers/redact` anonimiza correo, nombre, dirección IP y mensaje; `shop/redact` borra solicitudes, registros, configuración y la tienda; `app/uninstalled` borra las sesiones.
- **El formulario público está firmado.** Las seis rutas que apuntan al proxy pasan por `authenticate.public.appProxy()`, que valida la firma que Shopify añade a la petición. No se puede invocar desde fuera de la tienda.
- **Todas las consultas están acotadas por tienda.** Listado, ficha, cambio de estado, importes, estadísticas y las dos búsquedas del formulario público filtran por `shopId`. El fallo más grave del proyecto hermano —dos comercios distintos viéndose las solicitudes por compartir el número `#1001`— aquí no existe.
- **No hay SQL escrito a mano.** Todo pasa por Prisma con parámetros. Ni una consulta concatenada, ni un `ORDER BY` construido con datos del usuario.
- **La consulta a GraphQL no se puede manipular.** La referencia del pedido se valida contra `^[A-Za-z0-9_-]{1,40}$` antes de entrar en el filtro `name:`.
- **No hay `eval`, ni `exec`, ni `innerHTML`, ni `dangerouslySetInnerHTML`** en ninguna parte. El HTML del formulario público se monta escapando todas las variables, y el escapado incluye `{`, `}` y `%`, que es lo que hace falta porque esa página se sirve como Liquid y Shopify la procesa antes de entregarla.
- **Los colores de marca ya se validaban al pintarlos** con `safeColor()`. El fallo de inyección de JavaScript por los colores que tenía el proyecto hermano aquí ya estaba resuelto.
- **No hay secretos en el paquete.** No hay `.env`, ni claves escritas en el código, ni guiones de despliegue con contraseñas. El `client_id` del `shopify.app.toml` es la clave pública de la aplicación, no un secreto. El fichero `prisma/dev.sqlite` que viaja en la carpeta se ha inspeccionado byte a byte: **contiene el esquema y ninguna fila**, ni un testigo de sesión ni un correo.
- **Nadie define cabeceras CORS**, así que ningún origen externo está autorizado a llamar a la aplicación. Es lo correcto para lo que hace.

Lo que se ha arreglado en esta revisión

1. Cualquiera podía ver el pedido de otra persona y desistir en su nombre. Es el fallo grave de esta versión. En `app/lib/proxy-handler.server.ts`, la función `fetchOrderLines()` buscaba el pedido por su número, intentaba encontrar el que además tuviera el correo indicado y, si no lo encontraba, **se quedaba con el primero de la lista igualmente**:

```
const node = matchByName.find(...coincide el correo...)?.node || matchByName[0]?.node;
```

Ese `|| matchByName[0]?.node` anulaba la comprobación entera. En la práctica: se escribe `1001` —el primer pedido de casi cualquier tienda— y un correo inventado, y la página enseña el nombre real del pedido y **todas sus líneas de producto con sus cantidades**; y el envío del formulario crea una solicitud de desistimiento legítima sobre el pedido de un tercero. Con probar `1001`, `1002`, `1003` se recorre el catálogo de compras de la tienda entera. Se ha eliminado el respaldo: ahora el correo tiene que coincidir con el del pedido, y además tiene que ser no vacío, para que un pedido sin correo —los creados a mano en el administrador— no case con nada.

2. Correos y números de pedido escritos en los ficheros de registro. El mismo fichero tenía seis llamadas a `console.error` que se ejecutaban en **cada** petición del formulario, no solo al fallar, y volcaban el correo del cliente, la consulta y, en una de ellas, el nombre y el correo de **todos** los pedidos devueltos por la búsqueda. Eso es un fichero de registro con datos personales dentro, en un servidor donde los lee cualquiera que tenga acceso, y sin ninguna política de borrado. Se han sustituido por una función `dbg()` que solo escribe si la variable de entorno `WK_DEBUG` vale `1` y que **nunca** escribe datos del cliente: solo cuántos pedidos coinciden y qué comprobación ha fallado.

3. Datos personales en el registro del RGPD. El manejador de `customers/data_request` escribía el identificador del cliente en el registro. Ahora solo anota cuántas solicitudes tiene. Es el mismo arreglo que se hizo en el proyecto hermano.

4. El estado de una solicitud se guardaba sin comprobar. `changeStatus()` recibía el valor de un desplegable y lo escribía tal cual. Un valor inventado —trivial desde la consola del navegador— se guardaba, y la ficha de esa solicitud dejaba de abrirse: la pantalla busca la etiqueta y el color en una tabla fija y se encuentra un `undefined`. Se ha añadido la lista blanca en el servicio, y la pantalla ahora enseña el error en un aviso rojo en vez de caerse con la página de excepción.

5. Los importes admitían cualquier cosa. `setRefundAmount()` recibía el resultado de un `parseFloat()` sobre texto libre y lo escribía sin mirar. Un campo vacío daba `NaN`, y se aceptaban importes negativos —con los que el total se dispara, porque la depreciación se resta—. Ahora los tres importes se validan como número finito y positivo, se redondean a dos decimales, y la nota de depreciación se recorta a dos mil caracteres.

6. El plan de facturación iba de un campo oculto directo a la API de cobro. `billing.request()` recibía `String(fd.get("plan"))` sin comprobar. Se comprueba ahora contra la lista cerrada de los tres planes.

7. El formulario público no tenía ni límites de longitud ni validación. Cualquier visitante de la tienda llega a él. No había tope en ningún campo —se podían meter megas de texto en las observaciones—, el motivo se guardaba sin comprobar contra la lista de motivos válidos y el correo de contacto no se validaba. Ahora todos los campos se recortan (190 caracteres nombre y correos, 40 el teléfono, 2.000 las observaciones, 4.000 el listado de productos), el motivo se valida contra la lista cerrada, el correo tiene que tener forma de correo y faltar un dato obligatorio devuelve un 400.

8. El envío repetido creaba solicitudes duplicadas. La página de búsqueda sí avisaba de que ya existía una solicitud, pero el envío del formulario no lo comprobaba: recargar o pulsar dos veces creaba dos filas. Ahora

se comprueba antes de crear, y contra el nombre **real** del pedido devuelto por Shopify, no contra el campo oculto que el cliente puede manipular. Si ya existe, se enseña la que hay con su estado.

9. Cabeceras anti-caché incompletas. Las cinco respuestas del proxy llevaban solo `Cache-Control: no-store`. Son páginas con el nombre, el correo y el estado de la solicitud de una persona concreta, servidas por detrás de la red de distribución de Shopify. Se han añadido las tres cabeceras completas — `Cache-Control` con `no-cache`, `must-revalidate`, `max-age=0`, `Pragma` y `Expires` — sin tocar el `Content-Type`, que tiene que seguir siendo `application/liquid` para que Shopify procese la página.

10. La página prometía al cliente un correo y un PDF que no existen. Dos frases —«recibirás copia por email y un PDF Anexo B descargable» y «Recibirás confirmación por email con el PDF Anexo B»— prometían al consumidor un justificante que el código no genera ni envía en ninguna parte. Se han reescrito para que digan lo que de verdad pasa. No es un fallo de seguridad, pero sí un incumplimiento delante del cliente, y el arreglo cuesta dos líneas.

11. Dos entradas del panel podían tumbar la pantalla. El campo «Días exactos» pasaba por `parseInt` sin comprobar y un valor no numérico llegaba como `NaN` a una columna de tipo entero, con lo que Prisma devolvía un 500 al guardar. Y los dos colores se guardaban sin validar, apoyándose solo en que la página pública los filtrara al pintarlos. Los tres se validan ahora también al guardar.

12. Código muerto que creaba filas basura. La función `_ignored()` de `app/services/desist.server.ts` no la llamaba nadie, pero si alguien la hubiera llamado habría dado de alta una tienda con el dominio `"noop"` en la base de datos. Eliminada.

Ficheros tocados: `app/lib/proxy-handler.server.ts`, `app/services/desist.server.ts`, `app/routes/webhooks.$.tsx`, `app/routes/app.requests.$id.tsx`, `app/routes/app.billing.tsx`, `app/routes/app.billing.callback.tsx`, `app/routes/app.settings.general.tsx`, `app/routes/app.settings.deadline.tsx`, `app/shopify.server.ts` y `app/lib/constants.ts`. Los diez pasan el analizador sintáctico de TypeScript y TSX.

Lo que queda pendiente

A. La mitad de la configuración no la aplica nadie. Es lo más importante de esta lista. Están escritas, y bien escritas, las funciones que calculan la fecha límite (`deadlineFor`, `isWithinDeadline`, `daysLeft`), la que decide si una línea está excluida (`isLineExcluded`) y la que detecta un pedido B2B (`isB2B`). **Ninguna se invoca desde ningún sitio.** También está definida la tabla de límites por plan (`PLAN_LIMITS`, con las 150 solicitudes mensuales del plan Starter) y la columna `requestsCount` de la tabla `Shop`, y tampoco las usa nadie: el plan no limita nada. Y la casilla «Activar botón de desistimiento» del panel no la lee el formulario público, así que desactivarla no lo apaga.

Es exactamente el fallo que no ve ninguna revisión que lea funciones de una en una: cada pieza es correcta, y el error está en la línea que nadie escribió. No se ha tocado porque conectarlas cambia el comportamiento de la aplicación de arriba abajo —hay que decidir qué se le enseña al cliente cuando está fuera de plazo, cuando su producto está excluido y cuando su pedido es de empresa— y eso es funcionalidad nueva, no un parche. Mientras tanto: **el comerciante tiene que comprobar a mano el plazo, las exclusiones y el carácter B2B de cada solicitud.**

B. Las suscripciones nunca se cobran. La línea `isTest: !session.shop.endsWith(".myshopify.com") ? false : true` dejaba el modo de prueba activado para **todas** las tiendas, porque el dominio interno de toda tienda

de Shopify termina en `.myshopify.com`. Es el mismo fallo del proyecto hermano. **No se ha cambiado el comportamiento**, porque afecta al dinero y a las pruebas en curso y es una decisión del usuario, no una corrección de seguridad. Lo que sí se ha hecho es sacarlo a una constante única, `BILLING_IS_TEST` en `app/shopify.server.ts`, usada a la vez en la petición del plan y en las dos comprobaciones —tienen que valer lo mismo o el plan contratado no se encuentra después—. Para empezar a cobrar de verdad basta con definir `SHOPIFY_BILLING_TEST=false` en el servidor y reiniciar. Sin esa variable, todo sigue exactamente como estaba.

C. Ni correo de confirmación ni PDF del Anexo B. No hay ninguna biblioteca de correo ni de PDF en las dependencias, y no hay una sola línea que envíe nada. Los textos que lo prometían ya se han corregido (arreglo 10), pero la funcionalidad sigue sin existir y los planes de pago la siguen anunciando.

D. El contador del icono flotante no funciona. `app/routes/count.tsx` reexporta el manejador del formulario, así que `/apps/desistimiento/count` devuelve la página HTML entera donde el JavaScript del tema espera un JSON con un número. El error se traga en silencio y el globo no aparece nunca. Y hay un efecto secundario que sí importa: **cada carga de página de la tienda con el icono activo y un cliente identificado dispara una consulta a la base de datos y el renderizado completo de la página del formulario**, para tirarla a la basura. Es un desperdicio de recursos del servidor proporcional al tráfico de la tienda. Arreglarlo bien exige decidir qué se cuenta y cómo se identifica al cliente —hoy las solicitudes se guardan por correo, y la columna `shopifyCustomerId` existe pero no se rellena nunca—, así que es trabajo de diseño, no un parche.

E. No hay límite de peticiones al formulario público. El control de duplicados que se ha añadido evita las filas repetidas del mismo pedido y el mismo correo, pero nada impide a un guion probar miles de combinaciones de número de pedido. Cada intento, además, consume una llamada a la API de administración de Shopify, que tiene cupo. Lo natural es resolverlo en el servidor web con `limit_req_zone` de nginx sobre la ruta del proxy.

F. Las dependencias no se han auditado en esta copia. No se ha ejecutado `npm install` ni `npm audit`, que era la condición del encargo. El `package.json` es de la misma generación que el del proyecto hermano —Remix 2.16, el SDK de Shopify 4.1, Vite 6—, donde `npm audit` encontró cuarenta y seis paquetes con vulnerabilidades conocidas, una crítica. Es **razonable suponer** que aquí ocurre lo mismo, pero no está comprobado y no se debe dar por confirmado. Es lo primero que hay que hacer en la próxima sesión, con el entorno levantado.

G. La base de datos es SQLite con la ruta escrita en el código. Ya explicado en el apartado 2. No es un fallo de seguridad, es un fallo de despliegue esperando a ocurrir: la primera vez que se redesplice el contenedor se pierden las solicitudes de todos los comercios.

H. La documentación de `docs/` es la del proyecto hermano. `INSTALL.md` explica cómo desplegar con `deploy/deploy.sh` y con un `.env.example` que **en esta copia no existen**, y se refiere a la carpeta `desistimiento-shopify`. Quien la siga tal cual se atascará en el paso 2. Hay que reescribirla o borrarla.

I. El `Dockerfile` usa `node:18-alpine` y el `package.json` exige Node 20.19 o superior. La imagen no sirve tal cual.

9. Ficha técnica

Identidad de la aplicación

Dato	Valor
Nombre	WithdrawalKit EU - EuroDesist
<code>client_id</code>	<code>bbaec75b31099cb6cab9533f7d912dd5</code>
URL de la aplicación	<code>https://desistimiento.gmartos.es</code>
Proxy público	prefijo <code>apps</code> , subruta <code>desistimiento</code> → <code>/apps/desistimiento</code>
Embebida	Sí (App Bridge + Polaris)
Distribución	App Store (<code>AppDistribution.AppStore</code>)
Autor	Ecom Experts · <code>ecomyseo@gmail.com</code> · AFL-3.0

Tablas (Prisma / SQLite)

Tabla	Para qué
<code>Session</code>	Sesiones de Shopify, gestionadas por el propio SDK, con testigos de refresco.
<code>Shop</code>	Una fila por tienda: dominio, fechas de instalación y desinstalación, plan, periodo de prueba y contadores.
<code>ShopConfig</code>	Toda la configuración de una tienda. Los textos legales, los datos del empresario y las listas de exclusión se guardan como JSON en columnas de texto, porque SQLite no tiene tipo JSON.
<code>DesistRequest</code>	La solicitud: pedido, cliente, estado, motivo, marca B2B y los tres importes.
<code>DesistRequestDetail</code>	Las líneas de producto de cada solicitud.
<code>DesistLog</code>	Auditoría: cada cambio de estado con origen, destino, nota, tipo de actor y fecha.
<code>WebhookSeen</code>	Identificadores de webhook ya procesados, para no repetir el trabajo con los reenvíos de Shopify.

Rutas

Panel del comerciante (todas con `authenticate.admin`): `/app` (panel principal), `/app/requests` (listado), `/app/requests/:id` (ficha), `/app/settings` con las cinco pestañas `general`, `deadline`, `exclusions`, `b2b`, `emails`, y `/app/billing` con su `/app/billing/callback`.

Formulario público (todas con `authenticate.public.appProxy`): `/apps/desistimiento` y su comodín, `/proxy` y su comodín, `/count`, el comodín de la raíz `/` y la raíz `/` cuando llega con el parámetro

`signature` . Las seis comparten el mismo manejador, `app/lib/proxy-handler.server.ts` , reexportado desde cada una porque Vite no resuelve el carácter `$` en una ruta de importación.

Autenticación y públicas: `/auth/*` , `/auth/login` , `/landing` , y `/` que redirige a `/app` si llega del administrador y a `/landing` si no.

Webhooks

Tema	Ruta declarada	Qué hace
<code>app/uninstalled</code>	<code>/webhooks/app/uninstalled</code>	Marca la desinstalación y borra las sesiones
<code>app_subscriptions/update</code>	<code>/webhooks/app_subscriptions/update</code>	Actualiza el plan de la tienda
<code>customers/data_request</code>	<code>/webhooks/gdpr/customers/data_request</code>	Reúne las solicitudes de ese cliente
<code>customers/redact</code>	<code>/webhooks/gdpr/customers/redact</code>	Anonimiza correo, nombre, IP y mensaje
<code>shop/redact</code>	<code>/webhooks/gdpr/shop/redact</code>	Borra todo lo de esa tienda

Todos llegan al mismo manejador `app/routes/webhooks.$$.tsx` , que verifica la firma HMAC y descarta los duplicados con `WebhookSeen` . Quedan además dos ficheros heredados de la plantilla oficial de Shopify (`webhooks.app.uninstalled.tsx` y `webhooks.app.scopes_update.tsx`) que hacen lo mismo por su cuenta.

Extensiones

Extensión	Tipo	Objetivos
<code>theme-floating-icon</code>	Tema	<code>floating-icon.liquid</code> (bloque incrustado, objetivo <code>body</code>) y <code>footer-link.liquid</code> (bloque de sección)
<code>customer-account-desist</code>	Interfaz de cuenta de cliente	<code>customer-account.order-status.block.render</code> y <code>customer-account.order-index.block.render</code>

Variables de entorno

Variable	Obligatoria	Para qué
<code>SHOPIFY_API_KEY</code>	Sí	Clave pública de la aplicación
<code>SHOPIFY_API_SECRET</code>	Sí	Secreto de la aplicación; firma sesiones y webhooks
<code>SHOPIFY_APP_URL</code>	Sí	URL pública; se usa en el retorno de la facturación
<code>SCOPES</code>	Sí	Lista de permisos separados por comas
<code>SHOP_CUSTOM_DOMAIN</code>	No	Dominio propio adicional admitido
<code>WK_DEBUG</code>	No	A <code>1</code> , activa el registro del formulario público (sin datos personales)
<code>SHOPIFY_BILLING_TEST</code>	No	A <code>false</code> , los planes se cobran de verdad. Por defecto, modo de prueba

Planes de facturación

Plan	Precio	Prueba
Starter	9,99 € / 30 días	14 días
Standard	19,99 € / 30 días	14 días
Pro	39,99 € / 30 días	14 días

Los límites que anuncian las tarjetas de precios —150 solicitudes al mes, 7 idiomas, eventos de Shopify Flow, webhooks salientes, estadísticas, exportación— **no están implementados**. Ver los apartados 2 y 8.

Registros

No hay carpeta de registros propia. Todo lo que la aplicación escribe va a la salida estándar del proceso de Node, y lo recoge el gestor de procesos del servidor. Conviene configurar una rotación por tamaño, porque no la hay.